Servomoteurs Série M17 - FICHE TECHNIQUE

Contrôle de mouvement tout-en-un, simple et abordable

De l'éducation à l'innovation



Introduction

Les servomoteurs de la série M17 sont des solutions de contrôle de mouvement tout-en-un qui intègrent un moteur, un pilote de moteur, un contrôleur de mouvement et un encodeur dans un seul boîtier compact. Ces servomoteurs disposent d'une interface de communication RS-485, permettant de chaîner plusieurs unités et de les contrôler à partir d'un seul point de connexion. Disponible en quatre modèles - M17-60, M17-48, M17-40 et M17-34 - la série offre des options flexibles pour répondre à des exigences spécifiques de couple et de taille tout en conservant des caractéristiques de contrôle homogènes. Chaque servomoteur de la série M17 est doté de fonctions de contrôle sophistiquées, notamment plusieurs modes de fonctionnement, des capacités d'auto-étalonnage et une surveillance d'état intégrée via des LED d'indication. Les moteurs peuvent être facilement contrôlés depuis n'importe quelle plateforme, y compris Mac, PC, Raspberry Pi ou Arduino (ne nécessitant qu'un adaptateur RS485 peu coûteux), ce qui les rend idéaux pour les applications éducatives comme industrielles. La série prend en charge un contrôle de position précis avec des profils de mouvement trapézoïdaux, un mode de contrôle en boucle fermée et une gestion complète des erreurs. Avec des dimensions de montage NEMA 17 standardisées, une large plage de tension (12-24V) et un protocole de communication robuste, les servomoteurs de la série M17 constituent une solution fiable et flexible pour les applications exigeant un contrôle de mouvement précis, de la robotique et des machines CNC aux équipements de test automatisés et aux instruments scientifiques.

Envoyez-nous vos commentaires

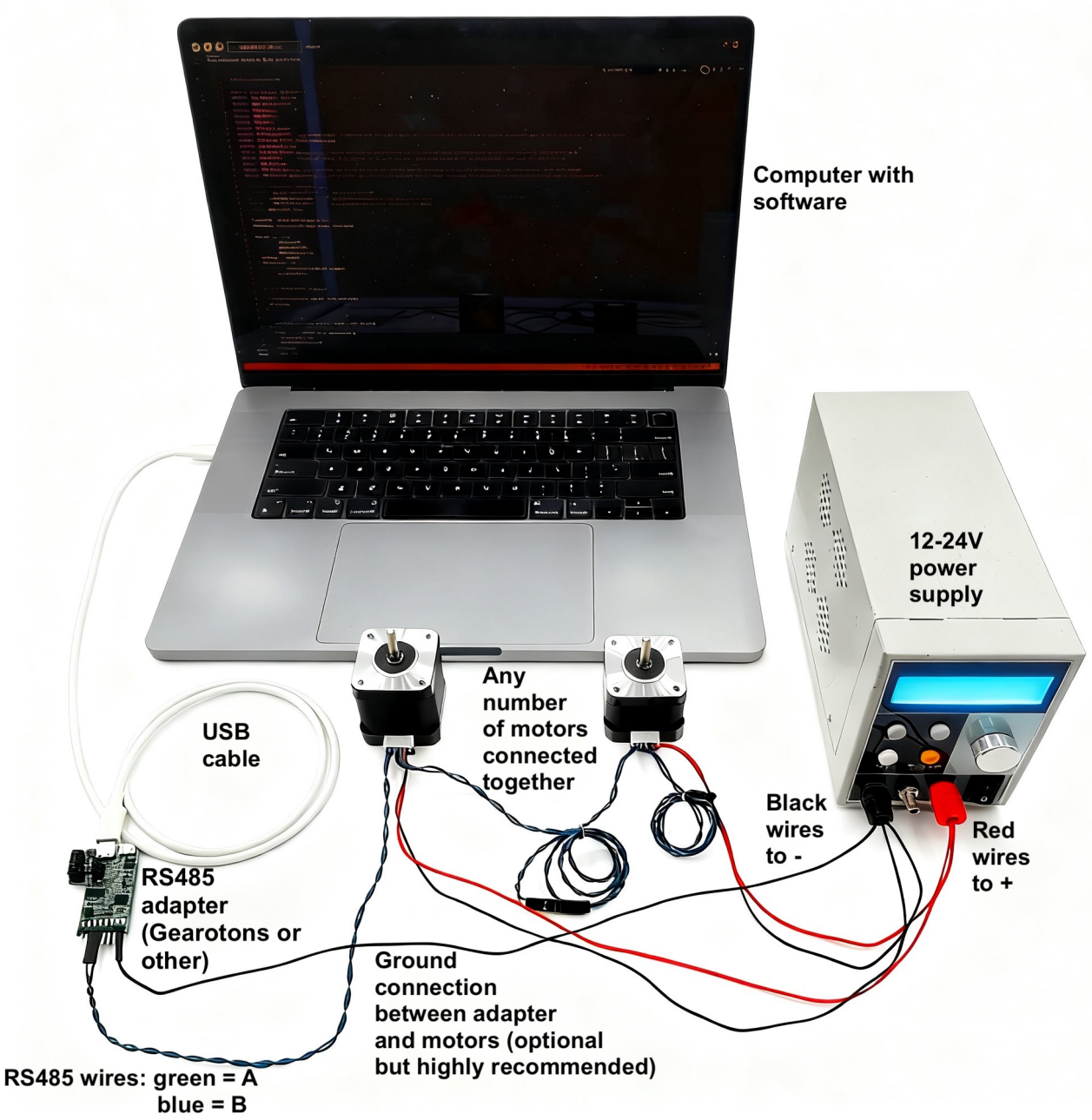
Si vous trouvez des erreurs dans ce document ou si vous avez des questions, veuillez nous en informer en vous rendant sur : tutorial.gearotons.com/feedback. Nous apprécierions vraiment vos suggestions sur la façon d'améliorer le produit ou la documentation. Merci beaucoup !

Visitez notre page de commentaires

Caractéristiques principales

- Haut niveau d'intégration combinant un moteur, un pilote de moteur, un système de contrôle de mouvement et un encodeur
- Contrôlez un nombre illimité de moteurs à partir d'un seul contrôleur simple via une interface de communication RS-485
- Contrôle par des commandes de haut niveau telles que "Enable mosfets" (activer les MOSFETs) et "Trapezoid move" (mouvement trapézoïdal), évitant le contrôle DIR/STEP aux contraintes temporelles critiques
- Format compact, de taille presque identique à celle d'un moteur pas à pas NEMA 17 aux mêmes spécifications (sans protubérances)
- Dimensions de montage NEMA 17 standardisées
- Large plage de tension (12-24V) pour des options d'alimentation flexibles
- Contrôle en boucle fermée de haute précision avec encodeur intégré et boucle de régulation PID fonctionnant à 32 kHz
- Bien plus économe en énergie en contrôle en boucle fermée qu'un moteur pas à pas conventionnel
- Protections intégrées contre les surintensités, les surtensions et les surchauffes
- La vitesse maximale peut atteindre 560 RPM
- Rapport couple/poids identique à celui d'un moteur pas à pas équivalent
- Compatible avec une grande variété d'interfaces et de matériels, tels que Raspberry Pi, Arduino, ESP32, Mac et PC
- Nous nous efforçons de fournir une excellente documentation et des tutoriels pour vous rendre opérationnel rapidement
- Nous fournissons une documentation adaptée aux IA au cas où vous souhaiteriez que votre IA préférée fasse tout le travail à votre place
- Convient à la robotique, aux machines CNC, à l'automatisation, aux instruments scientifiques, aux bancs de test, aux imprimantes 3D et à bien d'autres applications
- Disponible en différentes tailles afin que vous puissiez trouver le couple et le prix adaptés à votre application

Schéma de raccordement



Système d'unités

Les servomoteurs de la série M17 utilisent certaines unités internes afin d'effectuer efficacement les calculs liés au mouvement (en arithmétique entière). Il incombe au logiciel de contrôle de prendre en charge plusieurs unités de mesure pour les différentes grandeurs. Nos bibliothèques Python et Arduino gèrent automatiquement les conversions d'unités, vous permettant de travailler avec vos unités préférées. Voici les unités prises en charge pour chaque grandeur :

Grandeur	Unités disponibles
Temps	seconds, milliseconds, minutes, microseconds, timesteps
Position	shaft rotations, degrees, radians, encoder counts
Vitesse	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Accélération	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Courant	internal current units, milliamps, amps
Tension	volts, millivolts
Température	celsius, fahrenheit, kelvin

Guide de démarrage

Pour vous aider à démarrer avec votre servomoteur de la série M17, nous proposons un guide en ligne complet qui couvre tout, de la configuration initiale aux implémentations avancées du protocole. Ce guide comprend :

- Instructions de configuration pas à pas
- Documentation détaillée du protocole de communication
- Exemples de programmation et extraits de code
- Description des codes d'erreur
- Conseils de dépannage et bonnes pratiques



Cliquez ici pour notre guide de démarrage : tutorial.gearotons.com

LED d'indication et boutons

Le servomoteur possede deux LED d'etat (verte et rouge). La LED verte clignote lentement pour indiquer un battement de coeur (heartbeat), clignote rapidement pour indiquer que c'est le bootloader qui s'execute et non l'application, et s'allume aussi brievement pendant la reception d'un paquet, de sorte qu'elle scintille au rythme de la communication sur le bus. La LED rouge indique les codes d'erreur fatale en clignotant un certain nombre de fois.

Le servomoteur possède deux boutons étiquetés "Reset" (réinitialisation) et "Test". Le bouton Reset réinitialise le microcontrôleur interne et tous les états reviennent à leurs valeurs par défaut. Le bouton Test fait tourner le moteur. Appuyez brièvement pour le faire tourner dans un sens, et appuyez pendant plus de 0,3 seconde puis relâchez pour le faire tourner dans l'autre sens. Maintenez le bouton enfoncé pendant au moins 2 secondes puis relâchez pour faire passer le moteur en mode boucle fermée. Maintenez-le enfoncé pendant plus de 15 secondes puis relâchez pour que le moteur effectue un étalonnage sur lui-même. Notez qu'il tournera pendant l'étalonnage et doit pouvoir tourner librement pour que l'étalonnage réussisse ; retirez donc toute charge avant d'effectuer cette opération.

Aperçu du protocole de communication

Les servomoteurs de la série M17 utilisent le RS-485 pour la communication, ce qui permet de chaîner plusieurs moteurs. Chaque moteur possède une adresse unique de 64 bits attribuée en usine afin de pouvoir être adressé individuellement sur un bus RS-485. Chaque moteur peut également se voir attribuer un alias unique d'un octet (0-251) afin d'économiser des octets dans le paquet de communication tout en conservant un contrôle individuel similaire. Il existe aussi un alias de diffusion, qui est 255. Le protocole prend en charge un ensemble complet de commandes pour le contrôle de mouvement, la configuration et la surveillance d'état. La mise à jour du firmware via l'interface RS-485 est prise en charge. La vitesse de communication est fixée à 230400 baud.

Résumé de référence des commandes

Pour la source de référence à jour de toutes les commandes disponibles, vous pouvez consulter ce document.

 https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Vous pouvez également exécuter cette commande :

```
pip3 install servomotor   # run this just once to install the library and programs
servomotor_command -c
```

Cela affichera les informations contenues dans le fichier motor_commands.json sous une forme plus lisible et fournira des informations d'utilisation pour envoyer des commandes au moteur depuis la ligne de commande.

Contrôle de base

Commande	Description
Disable MOSFETs	Desactive immediatement les sorties du pilote de moteur (MOSFETs) ; la commande s'execute aussitot (elle n'est pas mise en file d'attente) et est idempotente.
Enable MOSFETs	Active les sorties du pilote de moteur (MOSFETs).
Reset time	Remettre à zéro l'horloge absolue en microsecondes de l'appareil ; cela vide également toute la file de mouvements et arrête immédiatement le moteur.
Emergency stop	Arrêter immédiatement le moteur : désactiver les MOSFETs, vider la file de mouvements, mettre la vitesse actuelle à zéro et ramener la consigne de position interne à la position actuelle.
Zero position	Définit la position actuelle comme position zéro (l'origine).
System reset	Reinitialise l'appareil.

Contrôle de mouvement

Commande	Description
Trapezoid move	Parcourt le deplacement signe donne pendant exactement la duree donnee, par rapport a la position a la fin du mouvement precedemment mis en file d'attente (pour une cible absolue, utilisez 'Go to position').
Go to position	Met en file d'attente un mouvement absolu fluide.
Homing	Effectue la prise d'origine du moteur en se deplacant jusqu'a ce qu'il rencontre une butee.
Go to closed loop	Passe en mode de controle de position en boucle fermee.
Move with acceleration	Met en file d'attente un segment a acceleration constante : l'acceleration est appliquee une fois par pas de temps pendant le nombre de pas de temps donne.
Move with velocity	Fait tourner le moteur a une vitesse constante pendant une duree fixe.
Multimove	Met en file d'attente jusqu'a 32 mouvements en une seule commande ; chaque mouvement est un segment d'acceleration ou un changement de vitesse instantane (selectionne pour chaque mouvement par le masque de bits moveTypes) execute pendant son nombre de pas de temps donne.

Configuration

Commande	Description
Set maximum velocity	Definit la vitesse maximale autorisee.
Set maximum acceleration	Definit l'acceleration maximale utilisee pour planifier tous les mouvements trapezoidaux mis en file d'attente par la suite ('Go to position', 'Trapezoid move', mouvements d'etalonnage) ; les elements deja presents dans la file d'attente ne sont pas affectes.
Start calibration	Execute un etalonnage complet du moteur.
Set maximum motor current	Definit le courant moteur maximal et le courant de regeneration maximal.
Set safety limits	Definit les limites de securite de position.
Test mode	Definit ou declenche un mode de test.
Set PID constants	Definit les constantes P, I et D du controleur en boucle fermee qui s'efforce de maintenir la trajectoire de mouvement.
Set max allowable position deviation	Définir l'écart maximal autorisé entre la position réelle du moteur (mesurée par les capteurs Hall) et la position commandée ; son dépassement déclenche l'erreur fatale 45 (ERROR_POSITION_DEVIATION_TOO_LARGE).
CRC32 control	Active ou desactive la couche CRC32 du protocole pour cet appareil, dans les deux sens.
Get PID constants	Lire un instantané cohérent des constantes P, I et D brutes configurées.

État et surveillance

Commande	Description
Get current time	Renvoyer le temps absolu de l'appareil sous forme d'un nombre non signé sur 64 bits de microsecondes réelles écoulées depuis la mise sous tension ou le dernier 'Reset time' ; il ne s'agit pas de l'heure civile.
Get n queued items	Renvoyer le nombre d'éléments présents dans la file de mouvements sous forme d'un seul octet entre 0 et 32 (la file contient au maximum 32 éléments).
Get hall sensor position	Obtient la position mesuree par les capteurs a effet Hall, qui est la position reelle de l'arbre.
Get status	Obtient les indicateurs d'etat du moteur et le code d'erreur fatale.
Control hall sensor statistics	Active ou desactive la collecte des statistiques des capteurs a effet Hall.
Get hall sensor statistics	Relit les statistiques recueillies a partir des trois capteurs a effet Hall analogiques.
Get position	Obtient la position commandee (desiree) actuelle, c'est-a-dire la position cible interne du profil de mouvement qui progresse a chaque cycle de controle.
Get comprehensive position	Obtient en une seule fois la position commandee, la position (mesuree) des capteurs a effet Hall et le compte de l'encodeur externe.
Get supply voltage	Obtient la tension mesurée de l'alimentation.
Get max PID error	Obtient l'erreur de position minimale et maximale observee par le controleur PID en boucle fermee depuis la derniere lecture, puis reinitialise les accumulateurs.
Get temperature	Obtient la temperature mesuree par un capteur analogique dedie situe sur le PCB du pilote de moteur (ni la puce du microcontroleur ni les enroulements du moteur).
Get debug values	Lire un instantané de nombreuses valeurs de diagnostic internes.
Get communication statistics	Obtient six compteurs d'erreurs de communication et, eventuellement, les reinitialise.

Gestion de l'appareil

Commande	Description
Time sync	Envoyer la valeur de l'horloge du maître au moteur afin qu'il puisse ajuster sa fréquence d'horloge.
Get product specs	Obtient deux constantes du produit necessaires aux calculs de mouvement.
Detect devices	Detecte tous les appareils connectes sur l'interface RS485.
Set device alias	Definit l'alias d'un octet de l'appareil, l'enregistre dans la memoire flash non volatile, puis reinitialise automatiquement l'appareil.
Get product info	Obtient les informations sur le produit : code produit (numero de modele), code de compatibilite du firmware, version materielle, numero de serie et ID unique, renvoyes sous la forme d'une charge utile de reponse de 28 octets.
Firmware upgrade	Ecrit une page de 2048 octets du firmware applicatif dans la memoire flash.
Get product description	Obtient une courte chaine de description du produit.
Get firmware version	Obtient la version du firmware, ou la version du bootloader si l'appareil execute actuellement le bootloader.
Ping	Envoie une charge utile de 10 octets et l'appareil repond immediatement avec les memes 10 octets (s'execute aussitot, non mise en file d'attente).
Vibrate	Faire vibrer le moteur (uniquement sur M1 ; sans effet sur M2, M17 et M23) en alternant rapidement la tension PWM en boucle ouverte entre +50 et -50 (amplitude fixe codée en dur), ou arrêter les vibrations.
Identify	Identifie un moteur en faisant clignoter rapidement sa LED verte : 30 clignotements d'environ 90 ms chacun, soit environ 2,7 secondes au total, apres quoi le clignotement normal du battement de coeur d'une seconde reprend.

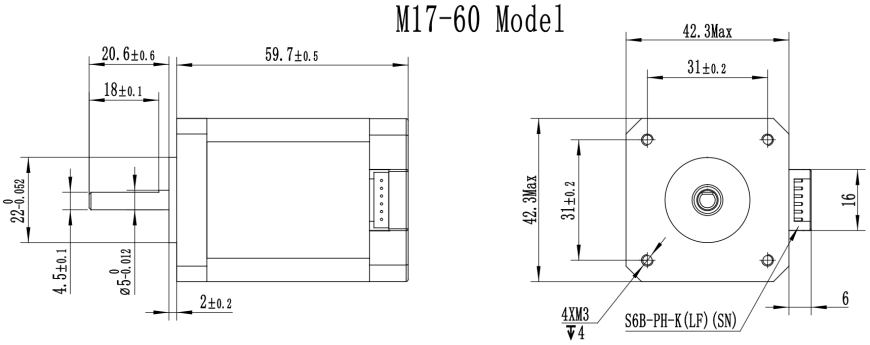
Autres

Commande	Description
Capture hall sensor data	Capture les donnees des capteurs a effet Hall et les retransmet en flux.
Read multipurpose buffer	Lit et efface le tampon de donnees polyvalent, qui contient au maximum un jeu de donnees a la fois.

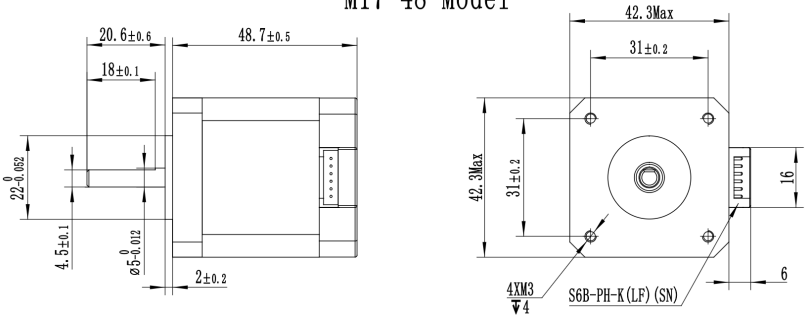
Spécifications mécaniques

Paramètre	M17-60	M17-48	M17-40	M17-34
Dimensions (L x l)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Hauteur	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Longueur de l'arbre	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Poids	470g	360g	285g	210g
Classe de protection	IP20	IP20	IP20	IP20

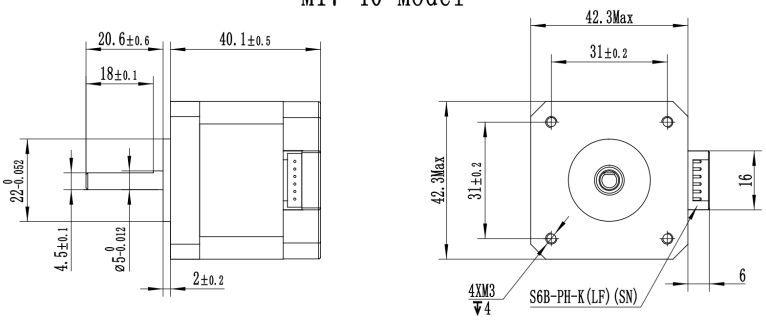
M17-60 Model



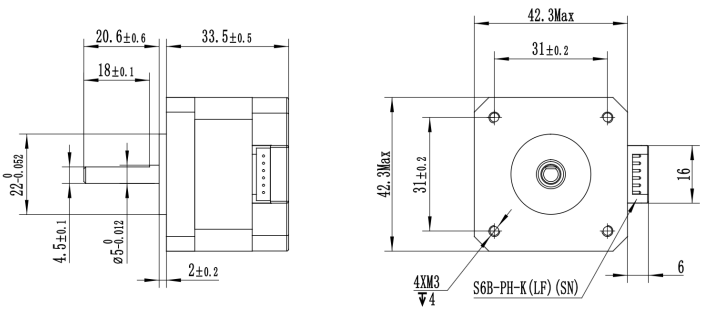
M17-48 Model



M17-40 Model



M17-34 Model



Spécifications techniques

Paramètre	M17-60	M17-48	M17-40	M17-34
Tension de fonctionnement	12-24V	12-24V	12-24V	12-24V
Couple nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Vitesse maximale	560 RPM	560 RPM	560 RPM	560 RPM
Courant maximal	1.1A	1.1A	1.1A	1.0A
Puissance nominale	26.4W	26.4W	26.4W	24.0W

Conditions de fonctionnement

Paramètre	Spécification
Température de fonctionnement	0C à +80C
Température de stockage	-20C à +60C
Plage d'humidité	20% à 80% RH (sans condensation)
Environnement d'installation	Utilisation en intérieur uniquement

Bibliothèque Python

Notre bibliothèque Python offre une interface complète pour contrôler les servomoteurs de la série M17. La bibliothèque gère tous les protocoles de communication de bas niveau ainsi que les conversions d'unités, facilitant l'intégration du contrôle de servomoteurs dans vos projets Python.

Installation

Installez la bibliothèque servomotor à l'aide de pip :

```
pip3 install servomotor
```

Exemple : contrôle de base du moteur

L'exemple suivant montre comment se connecter à un servomoteur, activer les MOSFETs et effectuer un mouvement trapézoïdal :

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                             # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                        #  Windows)
DISPLACEMENT_ROTATIONS = 1.0            # 1 rotation
DURATION_SECONDS = 1.0                  # 1 second
DELAY_MARGIN = 0.10                     # +10% wait margin because the motor's clock is not
                                        #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                        #  should prompt you for the serial port or it will use
                                        #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Caractéristiques principales

- Prise en charge de toutes les commandes du moteur
- Conversion automatique des unités pour le temps, la position, la vitesse, l'accélération, le courant, la tension et la température
- Gestion des erreurs et surveillance d'état
- Prise en charge de plusieurs moteurs sur le même bus
- Soyez opérationnel en quelques lignes de code seulement

Documentation

Pour la documentation complète de l'API et d'autres exemples, consultez :
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Bibliothèque Arduino

La bibliothèque Arduino pour les servomoteurs de la série M17 offre une interface facile à utiliser pour contrôler les moteurs depuis des cartes Arduino. La bibliothèque prend en charge la communication série matérielle et inclut toutes les fonctions essentielles de contrôle du moteur.

Installation

Installez la bibliothèque via le gestionnaire de bibliothèques de l'IDE Arduino :

- 1. Ouvrez l'IDE Arduino
- 2. Allez dans Croquis → Inclure une bibliothèque → Gérer les bibliothèques
- 3. Recherchez "Servomotor"
- 4. Trouvez la bibliothèque nommée "Servomotor by Gearotons" et cliquez sur Installer



Installation manuelle

Autrement, si vous êtes un expert, vous pouvez récupérer le code de la bibliothèque Arduino ici : https://github.com/tomrodinger/Servomotor_Arduino_Library

Exemple : contrôle de base du moteur

Cet exemple montre comment initialiser un moteur, activer les MOSFETs et effectuer un mouvement trapézoïdal :

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  system reset -> enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds)
//  -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.
// - Every command is checked with getError(). No command method returns a success
//   flag, and a motor that has latched a fatal error silently ignores everything
//   afterwards, so an unchecked sketch just stops moving with no symptom at all.
//   See "Checking for errors" in the API documentation.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))
#define POST_RESET_WAIT_MS 1500     // Keep the bus SILENT this long after a reset: the motor
                                    //  boots through a bootloader window, and any packet that
                                    //  arrives during it pins the motor in the bootloader.

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

// Returns true (and explains itself) if the previous command did not succeed.
// getError() is 0 on success, positive for a fatal error reported by the motor
// (look the number up in the Error Codes section of the documentation), and
// negative for a communication failure such as -1 = no reply within 1 second.
bool failed(Servomotor &motor, const char *what) {
  int e = motor.getError();
  if (e == 0) return false;
  Serial.print("[FAIL] ");
  Serial.print(what);
  Serial.print(" -> getError() = ");
  Serial.print(e);
  Serial.println(e > 0 ? "  (motor fatal error - see the Error Codes section)"
                       : "  (communication failure - check wiring, alias and power)");
  return true;
}

void setup() {
  Serial.begin(115200); // Console serial for debugging
                        // On ESP32-S3, set Tools > USB CDC On Boot > Enabled or this
                        // output never reaches the USB serial monitor.

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time.
  // These are host-side only - they send nothing to the motor.
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  // Start from a known-clean state: this also clears any fatal error left over
  // from a previous run, which is the usual reason a motor "stops working".
  motor.systemReset();
  if (failed(motor, "systemReset")) return;
  delay(POST_RESET_WAIT_MS);

  motor.enableMosfets();
```

```
  if (failed(motor, "enableMosfets")) return;

  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  if (failed(motor, "trapezoidMove")) return;

  delay(WAIT_MS);

  // A successful move command only means the move was accepted and queued. Faults
  // such as a stall or a position-deviation trip happen later, while it executes,
  // so check the motor's own status once the motion should be finished.
  getStatusResponse status = motor.getStatus();
  if (failed(motor, "getStatus")) return;
  if (status.fatalErrorCode != 0) {
    Serial.print("[FAIL] motor faulted during the move, error code ");
    Serial.println(status.fatalErrorCode);
    return;
  }

  motor.disableMosfets();
  if (failed(motor, "disableMosfets")) return;

  Serial.println("Move completed successfully.");
}

void loop() {
}
```

Notre mission

Chez Gearotons, nous croyons que l'avenir appartient à ceux qui comprennent l'intelligence artificielle et les systèmes d'automatisation et savent travailler à leurs côtés. Notre mission est de donner les moyens d'agir à la prochaine génération d'innovateurs en rendant la technologie avancée de contrôle de mouvement accessible, compréhensible et abordable pour les makers, les enseignants et les ingénieurs du monde entier. Nous nous engageons à faire tomber les barrières qui ont traditionnellement séparé les étudiants et les amateurs des outils d'automatisation de qualité professionnelle. En proposant des solutions intégrées qui éliminent la complexité sans sacrifier les capacités, nous rendons possibles des expériences d'apprentissage pratiques qui préparent les jeunes esprits à un monde où la créativité humaine et la précision des machines travaillent en harmonie. Notre conviction fondamentale est que chaque étudiant devrait avoir la possibilité d'expérimenter avec les briques de base de l'automatisation moderne - de la robotique et des systèmes CNC aux instruments scientifiques. Grâce à nos partenariats éducatifs et à notre approche open source, nous formons une génération qui ne se contente pas d'utiliser la technologie, mais qui la comprend et la façonne véritablement. Ensemble, nous bâtissons les fondations pour les innovateurs qui définiront un avenir extraordinaire.

Open Source

Nous croyons en un monde meilleur grâce à la technologie. Les logiciels, le firmware et les fichiers de conception des PCB sont disponibles ici :

https://github.com/tomrodinger/servomotor

Version de la fiche technique : 1.11 Date de publication : Oct. 29, 2026

(c) 2026 Toutes les spécifications peuvent être modifiées sans préavis.